**Quantitative Trading With Machine Learning**

**1. Problem Statement**

Financial markets generate large amounts of historical price and trading data. Traditional trading strategies often rely on manual analysis of price movements and market indicators, making it difficult to consistently identify useful patterns. Machine Learning provides an approach for automatically learning patterns from historical financial data and generating predictions that can support trading decisions.

The objective of this project is to develop a Machine Learning-based quantitative trading system that uses historical stock market data to predict future trading outcomes. The predicted outcomes are converted into trading signals and evaluated through historical backtesting.

The proposed system compares Logistic Regression and Random Forest classification models and evaluates them using both standard Machine Learning metrics and trading performance measures.

---

**2. Dataset Details**

The dataset consists of historical stock market observations containing information related to stock prices, trading volume, dates, and dividend-related variables.

The original dataset contained 17 columns. Important variables included:

- Stock identifier
- Date
- Open price
- High price
- Low price
- Close price
- Volume
- Periodic price changes
- Previous volume
- Days to next dividend
- Percentage return from the next dividend
- Target variable

After data cleaning, **720 usable observations** remained. The target variable was reasonably balanced:

| Target | Number of Samples |
|---|---|
| 0 | 358 |
| 1 | 362 |

The input data was divided chronologically into training and testing sets. The training set contained **570 observations**, while the test set contained **150 observations**.

A chronological split was used because financial prediction is a time-dependent problem. Randomly mixing past and future observations could result in information leakage.

---

**3. Approach**

The project follows a complete Machine Learning-to-trading pipeline.

**Data Preprocessing**

First, the historical dataset was loaded and inspected for missing values, data types, and target distribution. Unnecessary outcome-related columns were removed to avoid providing the model with information that would not be available at prediction time.

The date and categorical information were transformed into numerical representations. Numerical and categorical features were processed using a preprocessing pipeline.

After preprocessing, the training data contained approximately **41 features**.

**Machine Learning Models**

Two supervised classification algorithms were implemented.

**Logistic Regression:**
Used as a simple baseline model. It estimates the probability of the target class and provides an interpretable starting point for the trading prediction problem.

**Random Forest:**
A tree-based ensemble model consisting of multiple decision trees. It was selected because financial relationships can be nonlinear and Random Forest can capture interactions between different features.

**Prediction and Trading Signals**

After training, both models were used to predict the target class for unseen test data. The predictions were converted into trading signals.

A positive prediction represented a potential **Buy/Hold** decision, while a negative prediction represented a **No-Buy** decision.

The test period produced approximately:

- **111 Buy/Hold signals**
- **39 No-Buy signals**

---

**4. Implementation**

The implementation was developed in Python using Pandas, NumPy, Scikit-learn, Matplotlib, and Joblib.

The overall workflow was:

Historical Stock Dataset

↓

Data Cleaning

↓

Feature Selection & Engineering

↓

Categorical/Numerical Preprocessing

↓

Chronological Train-Test Split

↓

Logistic Regression + Random Forest

↓

Test Predictions

↓

Trading Signal Generation

↓

Backtesting

↓

Performance Evaluation

The models were evaluated using accuracy, precision, recall, and F1-score.

| Model | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|
| Logistic Regression | 47.33% | 39.42% | 71.93% | 50.93% |
| Random Forest | 44.00% | 37.84% | 73.68% | 50.00% |

Logistic Regression produced slightly better accuracy and F1-score, while Random Forest produced slightly higher recall.

---

**5. Backtesting Results**

To evaluate whether the predictions could be translated into a trading strategy, a simple backtesting procedure was implemented.

The strategy started with an initial capital of **₹100,000** and applied the generated trading signals to the corresponding test-period returns.

The approximate results were:

| Metric | Result |
|---|---|
| Initial Capital | ₹100,000 |
| Final Capital | ₹100,007.38 |
| Total Trades | 111 |
| Total Return | ≈ 0.01% |
| Maximum Drawdown | ≈ -1.28% |
| Sharpe Ratio | ≈ 0.0734 |

The backtest resulted in a very small positive return. However, the low Sharpe ratio indicates that the strategy did not provide strong risk-adjusted performance during the tested period.

---

**6. Conclusion**

This project demonstrates the complete application of Machine Learning to a quantitative trading problem. Historical stock data was processed, relevant features were prepared, classification models were trained, predictions were converted into trading signals, and the resulting strategy was evaluated using backtesting.

The experiments show that Machine Learning can be used to identify patterns in financial data and generate systematic trading signals. However, the relatively low prediction accuracy and small backtested return demonstrate that financial markets are highly noisy and difficult to predict using a limited set of historical features.

An important conclusion is that **good classification performance does not necessarily guarantee profitable trading performance**. Trading systems must also consider risk, transaction costs, market conditions, and portfolio management.

Future improvements could include larger datasets, technical indicators, hyperparameter optimization, LSTM/time-series models, transaction-cost modelling, portfolio optimization, and advanced risk-management techniques.

Overall, the project provides a foundation for building more advanced Machine Learning-based quantitative trading systems.